COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 7, 2026	**DATE SUBMITTED:**	October 9, 2026
NAME:	LOPEZ, ANDREI DION C. REALUBIT, SETH ANDREW I.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 11 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

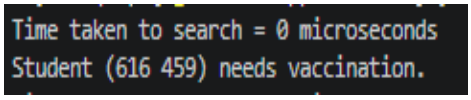
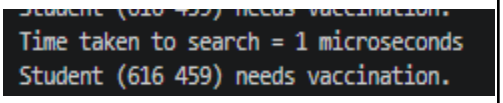
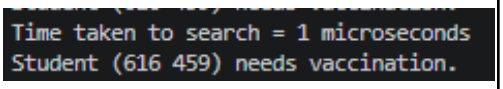
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the information above, the total number of comparisons in the worst case is: n^2 because the loop runs for n time and the "search" call costs m, leading the total to m * n.

Lopez Analysis:	**Realubit Analysis:**
The program becomes the worst case when it detects that the two arrays don't have common elements, this is because every element of y is compared against every element of x until the program ends. Since the program is n^2+n the values that are substituted for n (being 10, 20, and 30) the output becomes 110, 420, and 930 respectively, with the gaps increasing due to the equation exponentially increasing by n^2, making the runtime of the program longer the higher the values substituted to n are.	The worst case occurs when the two arrays do not share common values, thus every element of y is searched against all of x without returning early. The number of operations gives us the equation of n^2 + n, thus substituting the value of n = 10, 20 and 30 gives us 110, 420 and 930. The gaps between the values are not constant but rather, they grow by 200 each time. Thus, the runtime grows faster compared to the linear "search" (3n+3), leading the approach to become unusable in large arrays.

Table 1. ILO A

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 ms		The search took under 1 microsecond with the result of a student (616 459) needing vaccination.
10,000	1 ms		The search took 1 microsecond with students (616 459) needing vaccination.
100,000	1 ms		A student (616 459) was found needing vaccination in 1 microsecond.

Lopez Analysis:	**Realubit Analysis:**
Each output of the program showed little difference in runtime despite the input size increasing from 1000 to 100,000. Which matches the expected analysis speed of a binary search algorithm, with each iteration of the program dividing the program into two separate parts making the time complexity of the program O(logn). The program outputs microseconds for the runtime because it used “std::chrono::microseconds” forcing it to show only little difference in runtime.	The result shows that the search time barely changed even though the input size grew from 1000 to 100,000 entries, a 100x increase. This matched the theoretical analysis of binary search, which is O (log n) with each iteration halving the remaining range. One of the limitations I noticed was that we are using "std::chrono::microseconds" which is generally fine but using nanoseconds would give a better result and data for comparison as the runtime is affected by the operating system.

Table 2. ILO B

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

LOPEZ

Problem 1

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis:

The analysis of this program shows that the time complexity is $O(n^2)$, this is because the program loops n times for the first for loop, and n times for the second for loop. If n = 10 the time complexity would be $O(10^2)$.

Experimental Analysis:

```
1000 | comparisons=499500 | 2188 us
2000 | comparisons=1999000 | 8809 us
4000 | comparisons=7998000 | 34977 us
8000 | comparisons=31996000 | 142259 us
```

The experimental analysis shows us the exact same outcome as the expected outcome in the theoretical analysis of the program showing us that there is a gap for each growth of n, roughly being 4x with the runtime following the same rules.

Analysis and Comparison:
Both analysis shows that it follows the time complexity formula of n(n-1)/2 with only slight differences, as the runtime also depends on the performance of the hardware.

Problem 2

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis:
The theoretical analysis of this code snippet shows that the time complexity of the program is O(n) for rpower, and O(logn). Where brpower divides the exponent on each iteration of the program.
Experimental Analysis:

```
rpower  n=100 calls=101 800 ns
brpower n=100 calls=8 100 ns
rpower  n=200 calls=201 900 ns
brpower n=200 calls=9 200 ns
rpower  n=400 calls=401 9200 ns
brpower n=400 calls=10 200 ns
rpower  n=800 calls=801 27200 ns
brpower n=800 calls=11 200 ns
```

rpower calls n+1 times, where increasing the value of n also increases how many times its called, making its time complexity O(n). While brpower only gains 1 extra call when n increases, making the time complexity O(logn)

Analysis and Comparison:
Both experimental and theoretical analysis show the same time complexity, even when the values are increase, with the difference being that brpower uses a merge sort like algorithm that divides and conquers, calling itself recursively.

REALUBIT

Problem 1

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis:

The time complexity of the algorithm above follows an $O(n^2)$, because of the two for loops, the first and second for loops run for n times thus leading to n * n.

Experimental Analysis:

```
100 | comparisons=4950 | 24 us
200 | comparisons=19900 | 89 us
400 | comparisons=79800 | 351 us
800 | comparisons=319600 | 1400 us
```

The result of the program shows that it follows the formula of n(n-1)/2 for every n. Each time n doubles, the comparison grows roughly about 4x and the measured time follows the same pattern.

Analysis and Comparison:
Both the theoretical and experimental analysis follows the same formula of n(n-1)/2 but just in different forms. The theory predicts, while the measured time will depend on the hardware, which reaches the same growth rate.

Problem 2

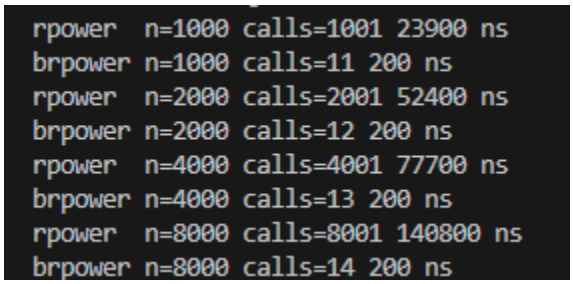
```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis:

rpower runs for O(n) with it reducing the exponent by 1 on each call, until it reaches the case of n = 0. Meanwhile the brpower halves the exponent on each call using n/2 when n is even and when n is odd it becomes (n-1)/2. It makes one recursive call per level and reuses the result y by squaring it instead.

Experimental Analysis:



The call count of rpower makes n+1 calls, so doubling n doubles the calls, and it roughly double each step, which becomes linear growth of O(n). Meanwhile, brpower gains only one extra call each time n doubles and the time complexity of O(log n).

Analysis and Comparison:

Both algorithms technically compute the same result even at lower values, the main difference they have is that brpower uses divide and conquer, reusing one recursive result instead of recomputing.

CONCLUSION	
LOPEZ	To conclude this activity, We were reintroduced to algorithm analysis and analysis of time complexity of programs, both of these topics are properly showcased in this activity, it also gave us a challenge on understanding how the theoretical analysis of the program compares to the experimental analysis of the program, where the main procedure showcases theoretical analysis of a program in ILOA, and experimental analysis in ILOB. With the supplementary activity combining both theoretical and experimental analysis and challenging us on comparing both.
REALUBIT	In this hands-on activity, we delved into the basics of algorithm analysis where the topics consisted more of into the time complexities. It also consisted of the range in the space complexity. We were greatly challenged due to the new methods and libraries which was shown evident in the ILO B procedure which consisted of the searching algorithm and returning the time it took to find the student. It also became dependent on the operating system on the runtime of the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Claude AI

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Code Assistance and explanation of concepts

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

AI was used for program refinement and correction suggestions for error messages that show up while running

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.